LAB 01 — SQL

Thực hành SQL trên Google Cloud Shell với SQLite

Introduction to Data Science | Trên lớp + Homework

1. Mục tiêu và chuẩn bị

- Sử dụng tài khoản Gmail, trình duyệt và Internet.
- Tạo CSDL quan hệ, khóa chính/khóa ngoại, CRUD, JOIN, GROUP BY, HAVING, subquery, index và kế hoạch truy vấn.
- Tất cả thực hành trên Cloud Shell miễn phí.

2. Nội dung

Cloud Shell và môi trường
Tạo bảng, thêm dữ liệu
CRUD và SELECT
JOIN, GROUP BY, subquery
Index, phân tích kế hoạch
Hoàn thiện và nộp

3. Đăng nhập Google Cloud Shell

1. Truy cập https://shell.cloud.google.com/ và đăng nhập Gmail cá nhân.
2. Chấp nhận điều khoản nếu xuất hiện; chọn Activate Cloud Shell (biểu tượng >_) để mở terminal.
3. Dùng chế độ mặc định có vùng $HOME, không chọn Ephemeral để giữ bài làm lâu dài.
4. Chạy các lệnh bên dưới. Không bật Billing hay tạo project có dịch vụ tính phí.

```
whoami
pwd
python3 --version
mkdir -p ~/data_science/sql_lab
cd ~/data_science/sql_lab
```

Cloud Shell thường có 5 GB lưu trữ trong $HOME và chịu hạn mức sử dụng. Tải bản sao code và CSDL về máy local để tránh mất dữ liệu nếu sau thời gian dài không đăng nhập.

4. Tạo SQL Console trong Cloud Shell

SQLite có sẵn trong thư viện chuẩn Python. Sao chép toàn bộ khối lệnh Bash dưới đây để tạo tệp sql_console.py và chạy chương trình.

```
cat > sql_console.py <<'END_SQL_PY'
import sqlite3
con = sqlite3.connect('sales.db')
con.execute('PRAGMA foreign_keys = ON')
print('SQL Console: end each statement with ; (EXIT to quit)')
buffer = ''
while True:
    try:
        line = input('SQL> ' if not buffer else '... ')
    except (EOFError, KeyboardInterrupt):
```

```
        break
    if line.strip().upper() == 'EXIT':
        break
    buffer += line + '\n'
    if sqlite3.complete_statement(buffer):
        try:
            cur = con.execute(buffer)
            if cur.description:
                print([x[0] for x in cur.description])
                for x in cur.fetchall(): print(x)
            else:
                con.commit()
                print('Rows affected:', cur.rowcount)
        except sqlite3.Error as e:
            print('SQL error:', e)
        buffer = ''
con.close()
END_SQL_PY
python3 sql_console.py
```

Nhập từng câu lệnh SQL và kết thúc bằng dấu ;. Ví dụ SELECT 1;. Chương trình thực thi một câu lệnh mỗi lượt. EXIT để thoát. Chạy lại python3 sql_console.py để mở database sales.db.

5. Mô hình dữ liệu và CREATE TABLE

Cửa hàng trực tuyến có bốn bảng: customers (1–N) orders, orders (1–N) order_items và products (1–N) order_items. Thực thi riêng từng câu CREATE:

```
CREATE TABLE customers (
 customer_id INTEGER PRIMARY KEY,
 name TEXT NOT NULL, city TEXT
);

CREATE TABLE products (
 product_id INTEGER PRIMARY KEY,
 name TEXT NOT NULL, category TEXT,
 price REAL CHECK(price >= 0)
);

CREATE TABLE orders (
 order_id INTEGER PRIMARY KEY,
 customer_id INTEGER NOT NULL, order_date TEXT,
 FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
 order_id INTEGER, product_id INTEGER,
 quantity INTEGER CHECK(quantity > 0),
 PRIMARY KEY(order_id,product_id),
 FOREIGN KEY(order_id) REFERENCES orders(order_id),
 FOREIGN KEY(product_id) REFERENCES products(product_id)
);
```

6. INSERT dữ liệu mẫu

Nhập riêng từng câu lệnh. Chỉ thực hiện một lần trên database mới.

```
INSERT INTO customers VALUES
(1,'An','Hanoi'),(2,'Binh','Hanoi'),(3,'Chi','Danang'),(4,'Dung','HCMC'),(5,'Ha','Hanoi');

INSERT INTO products VALUES
(101,'Laptop','Electronics',15000000),(102,'Mouse','Accessories',250000),(103,'Keyboard','Accessories',650000),(104,'
Monitor','Electronics',3500000),(105,'Headphones','Accessories',800000);
```

```
INSERT INTO orders VALUES (1001,1,'2026-10-01'),(1002,2,'2026-10-02'),(1003,1,'2026-10-05'),(1004,3,'2026-10-
06'),(1005,4,'2026-10-07'),(1006,5,'2026-10-08');

INSERT INTO order_items VALUES
(1001,101,1),(1001,102,2),(1002,103,1),(1002,104,1),(1003,105,2),(1004,101,1),(1005,102,3),(1006,104,2);
```

Kiểm tra số dòng bằng SELECT COUNT(*) FROM customers; và các bảng còn lại. Kỳ vọng: customers = 5, products = 5, orders = 6, order_items = 8.

7. Bài tập bắt buộc

Mã số	Yêu cầu
S01	Hiển thị tên và thành phố khách hàng
S02	Sản phẩm có giá > 1 triệu
S03	Sắp xếp sản phẩm theo giá giảm dần
S04	Thêm một khách hàng mới
S05	Tăng giá Mouse thêm 10%
S06	Xóa khách hàng vừa thêm (chưa có đơn)
S07	Liệt kê mã đơn và tên khách hàng bằng JOIN
S08	Tính tổng tiền từng đơn hàng
S09	Doanh thu từng khách hàng
S10	Khách hàng chi tiêu > 5 triệu (HAVING)
S11	Sản phẩm có giá cao hơn trung bình (subquery)
S12	Khách hàng có từ 2 đơn hàng
S13	Top 3 sản phẩm theo doanh thu
S14	Tạo index trên orders.customer_id
S15	Xem EXPLAIN QUERY PLAN trước và sau index

8. Ví dụ

Ví dụ S08 – tổng tiền theo đơn:

```
SELECT o.order_id, SUM(i.quantity*p.price) AS total
FROM orders o JOIN order_items i ON i.order_id=o.order_id
JOIN products p ON p.product_id=i.product_id
GROUP BY o.order_id ORDER BY total DESC;
```

Để học index, chạy câu EXPLAIN trước khi tạo index, sau đó tạo index và chạy lại:

```
EXPLAIN QUERY PLAN SELECT order_id FROM orders WHERE customer_id=1;

CREATE INDEX idx_orders_customer ON orders(customer_id);

EXPLAIN QUERY PLAN SELECT order_id FROM orders WHERE customer_id=1;
```

Đọc SCAN/SEARCH/USING INDEX. Với dữ liệu rất nhỏ không suy luận index luôn giúp truy vấn nhanh hơn.

9. Bài tập về nhà

- Mở rộng dữ liệu tối thiểu 100 khách, 30 sản phẩm, 500 đơn, 1.000 dòng đơn hàng; dữ liệu giả lập và khóa ngoại hợp lệ.
- S16: Doanh thu theo tháng; S17: Khách chưa có đơn; S18: Sản phẩm chưa từng bán.
- S19: Xếp hạng khách hàng theo doanh thu bằng RANK() hoặc DENSE_RANK().
- S20: So sánh kế hoạch thực thi sau khi mở rộng dữ liệu và tạo index.
- S21: Nêu khác biệt giữa SQL Server #table/##table/tempdb trong slide với SQLite TEMP TABLE.

10. Nộp bài, chấm điểm

Nộp file có tên StudentID_SQL.zip bao gồm sales.db, sql_console.py, answers.sql (ghi rõ -- S01…S21), chương trình tạo dữ liệu mở rộng, báo cáo PDF 2–3 trang và ảnh minh chứng.

Tiêu chí	Điểm
Cài đặt và tạo dữ liệu	2.0
CRUD và câu cơ bản	2.0
Truy vấn nâng cao	3.0
Index và giải thích	1.5
Bài mở rộng, báo cáo	1.5